Internal Management System - ASTERISCO TROVADOR

1. Introduction

1.1 Purpose

The purpose of this document is to outline the requirements for developing a CV Management System for an IT consultancy and software solutions company. The system will manage candidates, employees, business managers, and admins, providing a scalable solution for managing CVs, employee data, client relations, and more.

1.2 Scope

The platform will initially focus on CV management but will be extensible to include features such as invoice management, client management, and project tracking. It will be built using:

- **Backend**: Spring Boot
- **Frontend**: Angular
- **Authentication/Authorization**: Keycloak
- **Database**: PostgreSQL

---

2. Features and Functionalities

2.1 User Roles

The platform will support four user roles:

1. **Admin (Director)**:
    - Create and manage clients.
    - Manage user roles and permissions.
    - Oversee the system's overall functionality.
2. **Business Manager**:
    - Manage employee details (salary, benefits, promotions, etc.).
    - Track employee progress and performance.
    - Manage client-employee relations.
3. **Employee**:
    - View and update personal information.
    - Submit progress reports.
    - Access client-related details (if applicable).
4. **Candidate**:
    - Submit CVs and personal information via a secure, time-limited token-based URL.
    - Apply for specific job offers or spontaneous opportunities.
    - Create an account to track candidacy progress, respond to information requests, and view next steps in the process.

2.2 CV and Candidacy Management

- **Candidate Registration**:
  - Candidates receive a unique, time-limited token via email.
  - A secure URL directs them to the CV submission page.
  - Candidates can upload CVs and fill in personal information (e.g., name, contact details, skills, experience).
- **Candidate Accounts**:
  - Candidates can create an account to:
    - View all job applications they have submitted.
    - Check the status of their applications (e.g., pending review, interview scheduled, offer made).
    - Respond to additional information requests from the recruitment team.
    - View next steps in the application process (e.g., schedule an interview, provide documents).
- **Validation**:
  - Validate CV content before submission (e.g., required fields, file type/size limitations).
- **Storage**:
  - Save CVs and related data in a PostgreSQL database.
- **Employee Conversion**:
  - If a candidate is hired, their information is seamlessly transferred to the employee database.

2.3 Employee Management

- **Profile Management**:
  - Store personal details, salary, benefits, promotions, and job title.
- **Client Relations**:
  - Assign employees to clients.
  - Track client-employee relationships (e.g., project details, duration).
- **Performance Tracking**:
  - Enable managers to monitor employee progress.
  - Store performance reviews and feedback.

2.4 Administration

- **User Management**:
  - Admins can create, update, and delete users.
  - Assign roles and permissions to users.
- **Client Management**:
  - Add and manage client details (e.g., name, contact information, ongoing projects).

- **Audit Logs**:
    - Track all changes and actions performed by users for accountability.

2.5 Authentication and Authorization

- **Token-based Authentication**:
    - Candidates access the CV submission page using a secure, time-limited token.
- **Candidate Account Authentication**:
    - Candidates can log in to their accounts to track applications.
- **Keycloak Integration**:
    - Single Sign-On (SSO) for all users.
    - Role-based access control to enforce permissions.

2.6 Extensibility

- **Future Modules**:
    - Invoice Management: Track billing details for clients.
    - Project Management: Assign projects to employees and monitor progress.
    - Reporting: Generate detailed reports for admins and managers.

---

3. Non-Functional Requirements

3.1 Scalability

The system must support a growing number of users, clients, and CV submissions without performance degradation.

3.2 Security

- Use Keycloak for secure authentication and authorization.
- Encrypt sensitive data (e.g., passwords, salary information).
- Implement HTTPS for all network communications.

3.3 Usability

- Provide an intuitive user interface using Angular.
- Ensure the platform is responsive and mobile-friendly.

3.4 Performance

- Ensure fast response times for all user actions.
- Optimize database queries and API endpoints.

3.5 Maintainability

- Modular architecture for easy extension and maintenance.
- Use industry-standard practices for code quality and documentation.

---

4. System Architecture

4.1 Overview

- **Frontend**: Angular for creating a dynamic, responsive user interface.
- **Backend**: Spring Boot for business logic and APIs.
- **Authentication**: Keycloak for user authentication and authorization.
- **Database**: PostgreSQL for data storage.

4.2 Component Diagram

- **Frontend**:
  - Angular SPA
  - Communicates with the backend via REST APIs.
- **Backend**:
  - Spring Boot services.
  - Handles business logic and database interactions.
- **Database**:
  - PostgreSQL for storing structured data (e.g., users, CVs, clients).
- **Authentication**:
  - Keycloak for managing users and roles.

---

5. User Stories

5.1 Candidate User Stories

1. As a candidate, I want to receive a secure link to submit my CV.
2. As a candidate, I want to create an account to track my job applications.
3. As a candidate, I want to view the status of my applications.
4. As a candidate, I want to respond to additional information requests.
5. As a candidate, I want to view next steps in the application process.

5.2 Employee User Stories

1. As an employee, I want to view and update my personal details.
2. As an employee, I want to view my assignments and progress reports.

5.3 Business Manager User Stories

1. As a manager, I want to manage employee details (e.g., salary, benefits).
2. As a manager, I want to track client-employee relations.

5.4 Admin User Stories

1. As an admin, I want to create and manage user roles.
2. As an admin, I want to add and manage client details.

---

6. Data Model

6.1 Database Tables

1. **Users**:
    - Fields: ID, name, email, role, password, etc.
2. **Candidates**:
    - Fields: ID, name, email, CV, status, etc.
3. **Employees**:
    - Fields: ID, name, salary, benefits, manager_id, etc.
4. **Clients**:
    - Fields: ID, name, contact_info, etc.
5. **Applications**:
    - Fields: ID, candidate_id, job_id, status, steps_completed.

---

7. Technology Stack

- **Frontend**: Angular
- **Backend**: Spring Boot
- **Database**: PostgreSQL
- **Authentication**: Keycloak

---

8. Future Enhancements

1. **Invoice Management**:
    - Track payments and billing details for clients.
2. **Reporting**:
    - Generate detailed reports for employees, clients, and projects.

---

9. Deployment Plan

- **Environment**:
    - Development: Local environment with Docker containers.
    - Production: Cloud-hosted solution with CI/CD pipelines.